Red Hat | Red Hat Satellite | Upgrade Guide

Upgrading Satellite 6.18 to 6.19

Version numbering explained, connected and disconnected paths, troubleshooting, and an Ansible playbook to automate it.

Understanding Satellite Version Numbers

Satellite versions follow an `X.Y.Z` scheme. For `6.18.5`:

Field	Value	Meaning
X	6	Major version
Y	18	Minor / "y-stream" release — where new features land
Z	5	"z-stream" release — the maintenance counter

What the Z (the "5") represents: the z-stream is an aggregation of errata, carrying qualified Critical and Important security advisories (RHSAs) and Urgent or selected High-priority bug-fix advisories (RHBAs). It is maintenance and security fixes — **not** new features. So `6.18.5` means "Satellite 6.18 with five accumulated rounds of maintenance and security fixes." New functionality is held for the next y-stream (6.19).

A useful distinction:

- **z-stream update** (e.g. 6.18.4 → 6.18.5): patching within a release. Uses `satellite-maintain update`.
- **y-stream upgrade** (e.g. 6.18 → 6.19): moving to the next feature release. Uses `satellite-maintain upgrade`. **This guide covers the y-stream upgrade.**

> **Note on Lightspeed/Insights content:** the advisor and vulnerability *rules* update as content the on-premises IOP consumes; they are not tied to the

Satellite z-stream number. Patching from `.4` to `.5` is Satellite maintenance, not a Lightspeed rules refresh.

Upgrade Path and Compatibility

- **Direct path:** you can upgrade from 6.18 directly to 6.19 (a single y-stream hop). No intermediate version is required.
- **N-1 policy:** the Satellite Server can only be upgraded from the immediately preceding minor version. 6.18 → 6.19 satisfies this.
- **Capsule compatibility:** after upgrading the Satellite Server to 6.19, Capsules at **6.18 and 6.17** remain fully functional. You can upgrade Capsules later, in separate maintenance windows, rather than all at once.
- **Timing:** on average installations the Satellite Server upgrade takes roughly 30 minutes; large installations can take 1 to 2 hours. Each Capsule takes about 15 to 30 minutes.

Before You Begin

Run as root. Every command in this guide (`satellite-maintain`, `subscription-manager`, `podman login`, `dnf`) requires root privileges. Either run them as the `root` user (the `#` prompt convention), or prefix each one with `sudo`. The accompanying Ansible playbook handles this automatically via privilege escalation, so it is run as a normal user with sudo rights.

1. **Read the 6.19 release notes** and run the Satellite Upgrade Helper on the Customer Portal for instructions matched to your exact version.
2. **Confirm you are on the latest 6.18 z-stream.** Being current on 6.18 before the y-stream upgrade avoids known issues:

```
bash satellite-maintain update run
```

1. **Take a backup.** The upgrade runs `satellite-installer`, which overwrites installer-managed files. A current backup is your rollback path:

`bash satellite-maintain backup offline /var/backup/satellite-pre-6.19`

1. **Use tmux.** The upgrade is lengthy; run it inside `tmux` so a dropped SSH session does not interrupt it. Progress is also logged to `/var/log/foreman-installer/satellite.log`.

`bash satellite-maintain packages install tmux tmux`

1. **(Lightspeed) Confirm registry authentication.** `satellite-maintain` pulls the updated Lightspeed container images during the upgrade, so ensure the registry login is in place at the path the tool expects:

`bash podman login --authfile /etc/foreman/registry-auth.json registry.redhat.io`

1. **(DNS/DHCP) Preserve manual edits.** If you hand-edited DNS or DHCP configuration files, note them. `satellite-installer` overwrites the files it manages. You can preview changes with `satellite-installer --noop`.

Upgrading the Satellite Server

Perform these steps on the Satellite Server, inside your `tmux` session.

Step 1 — Enable the 6.19 maintenance repository

```
› _
subscription-manager repos --enable satellite-maintenance-6.19-for-rhel-9-x86_
```

This must be done before `self-upgrade`, and the system must be registered (you cannot enable Red Hat repositories on an unregistered system). The remaining 6.19 content repositories are enabled automatically during the upgrade run.

Step 2 — Upgrade the satellite-maintain tooling

For 6.19, update the maintenance tool with the self-upgrade command (this replaces the older `dnf`-based step from previous versions):

```
›_
satellite-maintain self-upgrade
```

If `self-upgrade` reports that only the `satellite-maintain` / `rubygem-foreman_maintain` packages were updated, run it once more so the refreshed tooling is fully in place before the check.

Step 3 — Run the pre-upgrade check

```
›_
satellite-maintain upgrade check -y
```

Your build of `satellite-maintain` derives the target version automatically from the enabled `satellite-maintenance-6.19` repository, so **no version argument is passed** (this build does not accept `--target-version`). The `-y` answers the confirmation prompts.

On first run, `satellite-maintain` configures Hammer CLI and prompts once for the **Satellite admin password** (the `admin` web UI account, not a system password). It saves this to `/etc/foreman-maintain/foreman-maintain-hammer.yml`, so subsequent runs do not prompt. Review the output and resolve every flagged condition before proceeding.

Step 4 — Run the upgrade

```
›_
satellite-maintain upgrade run -y
```

Again, no version argument: the target is taken from the enabled maintenance repository. This stops services, updates packages, runs `satellite-installer`, applies database migrations, pulls the updated Lightspeed container images, and restarts services. If you lose your shell, check `/var/log/foreman-installer/satellite.log` for progress.

Step 5 — Reboot if required

```
›_
dnf needs-restarting -r || systemctl reboot
```

Synchronizing 6.19 Content

After the Server is on 6.19, refresh and sync so clients receive current content:

1. **Refresh the subscription manifest** (resolves repository synchronization issues after the upgrade): **Content > Subscriptions > Manage Manifest > Refresh.**
2. **Sync repositories** as needed: **Content > Sync Status.**

Disconnected (Air-Gapped) Upgrades

If the Satellite Server cannot reach the Red Hat CDN, the upgrade follows the same `satellite-maintain` flow, but with two differences: content comes from local media instead of the CDN, and the repository-validation checks are skipped because they assume CDN connectivity.

1. Make the 6.19 content available locally

Obtain the Satellite 6.19 binary DVD/ISO images from the Customer Portal (on a connected system) and transfer them to the disconnected Satellite. Mount them and point local repositories at the mounted content, so the `satellite` and `satellite-maintenance` packages for 6.19 are resolvable without the CDN. The base RHEL 9 BaseOS/AppStream content must also be available locally (ISO or internal mirror).

2. Make the Lightspeed container images available locally

The connected upgrade pulls updated IOP/Lightspeed images from `registry.redhat.io` automatically. In a disconnected environment those images must be present locally first — either bundled on the ISO or pre-pulled and imported into the Satellite's container

storage before the upgrade runs. If the images are not available locally, disable the advisor temporarily or expect the IOP step to fail closed.

3. Run the upgrade with the disconnected whitelist

The pre-flight check and upgrade run skip the CDN-dependent repository steps:

```
›_
# Pre-flight check (disconnected)
satellite-maintain upgrade check -y \
  --whitelist="repositories-validate,repositories-setup"

# Run the upgrade (disconnected)
satellite-maintain upgrade run -y \
  --whitelist="repositories-validate,repositories-setup"
```

> If the upgrade fails due to missing or outdated packages, download and install those dependencies manually from the ISO, then re-run. Run the command from a directory **without** a `config/` subdirectory (e.g. `/root`) to avoid a scenario-not-found error.

Which path applies here

Worth confirming with the team: if the Satellite Server has the outbound access opened during the POC (to `cdn.redhat.com` and `subscription.rhsm.redhat.com`), the **connected** path above is simpler and is what to use. The disconnected path is only required if the Server genuinely has no route to the CDN. The two are not mixed — pick based on whether the Server itself can reach Red Hat.

Upgrading Capsule Servers

Because 6.19 Satellite supports 6.18 and 6.17 Capsules, this can happen later, in its own maintenance window. On each Capsule:

```
>_
# Enable the 6.19 Capsule maintenance repository
subscription-manager repos --enable satellite-maintenance-6.19-for-rhel-9-x86_

# Upgrade the maintenance tooling
satellite-maintain self-upgrade

# Pre-upgrade check
satellite-maintain upgrade check -y

# Run the upgrade (use tmux)
satellite-maintain upgrade run -y
```

> If you made manual DNS or DHCP edits on the Capsule, restore them from your backups after the upgrade.

Post-Upgrade Tasks

1. **Verify health:**

`bash satellite-maintain health check satellite-maintain service status`

1. **Confirm the version:**

`bash rpm -q satellite --qf '%{VERSION}\n'`

1. **Review templates.** If you cloned default provisioning templates, check whether the defaults changed during the upgrade and update your clones to match. Going forward, prefer custom provisioning snippets over cloning whole templates.
2. **(Lightspeed) Refresh host registration.** After upgrading to 6.19, have managed hosts re-register their Lightspeed client so analysis continues uninterrupted:

`bash insights-client --register --force`

You can run this across the fleet with Satellite's remote execution.

1. **Confirm Lightspeed services** are healthy: **Red Hat Lightspeed > Recommendations** and **Vulnerability** should populate as before.

Troubleshooting

`container-podman-login` check fails during the pre-upgrade check

The pre-upgrade check includes a step, `Check whether podman needs to be logged in to the registry` (`container-podman-login`), that verifies Satellite can authenticate to `registry.redhat.io` to pull the 6.19 Lightspeed/IOP images. In environments where outbound access to the registry is filtered (proxy, firewall, or CDN restrictions), this step can fail as a **false positive** even when authentication is actually valid:

```
›_
Scenario [Checks before upgrading] failed.
The following steps ended up in failing state:
  [container-podman-login]
```

First, verify the login is genuinely working:

```
›_
podman login registry.redhat.io          # should report credentials already v
podman login --get-login registry.redhat.io   # should return your registry us
```

If `--get-login` returns your username, the authentication is valid and the check is a false positive. You can safely whitelist that one step, exactly as the error message suggests:

```
›_
satellite-maintain upgrade check -y --whitelist="container-podman-login"
satellite-maintain upgrade run   -y --whitelist="container-podman-login"
```

During the run, watch for the `Update IoP containers: [OK]` line — that is the actual image pull succeeding, which confirms the whitelisted check was indeed a false positive and the Lightspeed containers updated correctly.

> **Only whitelist after verifying the login.** If `podman login --get-login` does **not** return your username, the failure is real: re-establish authentication (`podman login --authfile /etc/foreman/registry-auth.json registry.redhat.io` and `podman login registry.redhat.io`) before

proceeding. Whitelisting a genuinely broken login will let the upgrade start but the IOP container update will then fail.

`--target-version` is not recognized

If `satellite-maintain upgrade check --target-version 6.19` returns `Unrecognised option '--target-version'`, your build derives the target from the enabled maintenance repository instead. Run `satellite-maintain self-upgrade` (twice if it only updated the tooling on the first pass), then use the no-argument form: `satellite-maintain upgrade check -y`.

Upgrade prompts for a password

On first run, `satellite-maintain` configures Hammer CLI and prompts once for the **Satellite admin password** (the `admin` web UI account). It is saved to `/etc/foreman-maintain/foreman-maintain-hammer.yml` and will not prompt again. For automated runs, pre-seed that file so the upgrade does not hang.

Automating the Upgrade with Ansible

The accompanying playbook (`upgrade_satellite_6.19.yml`) orchestrates this entire flow with pre-flight safety: it verifies the current version satisfies the N-1 rule, confirms registration, takes an offline backup, enables the target repositories, self-upgrades the tooling, runs the pre-flight check, then the upgrade, and finally verifies health and reports the new version.

```
›_
# Connected (default)
ansible-playbook -i inventory.ini upgrade_satellite_6.19.yml --ask-vault-pass

# Disconnected / air-gapped (6.19 content already staged from ISO locally)
ansible-playbook -i inventory.ini upgrade_satellite_6.19.yml --ask-vault-pass
  -e satellite_connected=false
```

The playbook handles connected and disconnected modes from the same file via the `satellite_connected` variable, applying the `--whitelist` flags automatically in

disconnected mode. It deliberately does **not** upgrade Capsules — those are handled separately, in their own maintenance windows, after the Server is confirmed healthy.

> Because the upgrade is long-running, the playbook installs `tmux` and relies on `satellite-maintain`'s own resumability. As with any major upgrade, run it against a host you have a current VM snapshot or backup of.

Quick Reference

```
>_
Pre-flight      satellite-maintain update run          # be current on 6.18.z f
                satellite-maintain backup offline <dir>
                tmux

Upgrade         subscription-manager repos --enable satellite-maintenance-6.19-
                satellite-maintain self-upgrade                    # run twice
                satellite-maintain upgrade check -y                # target der
                satellite-maintain upgrade run   -y
                reboot if required

Post            satellite-maintain health check
                manifest refresh + content sync
                insights-client --register --force   (on hosts)
                upgrade Capsules later (6.17/6.18 stay compatible)
```

This guide reflects the connected upgrade path. For a fully disconnected Satellite, the sequence is the same but content is imported from media rather than synced from the CDN. Always confirm against the official "Upgrading connected Red Hat Satellite to 6.19" documentation at docs.redhat.com for your environment.